

AIOps Module 3 — Infrastructure & Containerization

Raghav Iyengar — IIT Madras — <https://github.com/DA24B020/aiops-as2>

Q1 — Single-stage vs. multi-stage Docker

File	Base image	Size
q1/Dockerfile.naive	python:3.11	1599.2 MB
q1/Dockerfile	python:3.11-slim	445.9 MB
Reduction: 1153.4 MB (72.1%), with identical API responses.		

Why the multi-stage image is smaller

The base accounts for most of it: `python:3.11` is 1.1 GB before anything is installed, against 133 MB for `python:3.11-slim`. The runtime stage adds back one package, `libgomp1`, which scikit-learn's wheels need. Three further things stay in the builder, all confirmed by running a shell inside each image: `gcc`, at `/usr/bin/gcc` in the naive image and absent from the other; pip's 85 MB wheel cache, absent because the venv install uses `--no-cache-dir`; and the training path — `import pandas` prints 2.2.3 in one and raises `ModuleNotFoundError` in the other. Only `main.py`, the 16 KB `model.joblib` and `/opt/venv` cross the `COPY --from=builder` boundary.

A control build (`q1/Dockerfile.control`) — single-stage on the slim base, 976.6 MB — splits the saving into **622.6 MB from the base swap** and **530.7 MB from the build/runtime separation**.

Q2 — Docker Compose with a Redis cache

Services and networking

`q2/docker-compose.yml` declares `api` (built from `q1/Dockerfile`) and `cache` (`redis:7-alpine`). The API is given `REDIS_HOST: cache` — the service name, not an address — and `depends_on: service_healthy` holds it back until Redis answers a ping. From inside the container, `socket.gethostbyname('cache')` returns `172.18.0.2` — an address nowhere in my configuration, because Compose runs an embedded DNS server on the project network.

Cache behaviour and measured speed-up

`/predict` is read-through, keyed on `spam:v1`: plus the SHA-256 of the input. A miss computes the label and writes it with `SETEX` at a 300s TTL; a hit is one `GET`. The body stays exactly `{"label": ...}`, with cache state reported in the `X-Cache` header. After a `FLUSHALL` the first request returned `MISS` and the identical second `HIT`, both with the same label.

Over $N=50$ each way: server-side **2.759 ms** → **0.805 ms (3.43×)**, end-to-end **7.863 ms** → **6.063 ms (22.9%)**. The end-to-end gain is smaller because roughly 5 ms of every request is fixed overhead the cache cannot touch.

Compose vs. Kubernetes

A Dockerfile describes one image; it cannot express a second service, the network between them, or their start-up order. Compose solves exactly that, on a single host. Kubernetes works a level above: scheduling pods across a cluster and reconciling declared state. Compose has no scheduler and no notion of a second machine.

Q3 — Indexed Job for parallel shard validation

Choosing parallelism: 4

The stated cluster is $2 \times 2 = 4000$ m. `validate_shard.py` is single-threaded, so one pod cannot exceed one core and four cores cap throughput however many pods are admitted. minikube's system pods already request 800–1000 m, leaving about 3000 m. At request = limit = 500m, parallelism 4 costs 2000 m — two

Pods per node, with headroom. Above four, CPU-bound pods would outnumber the cores for no gain.

Evidence that it was reached

A watcher polling `kubectl get pods -o wide` every three seconds recorded a maximum of **4 concurrently Running pods**. Indices 0–3 ran together and reached **Completed** before 4–7 were created — exactly $\lceil 8/4 \rceil = 2$ waves.

Collecting results through the API

Each pod prints a **RESULT** JSON line to stdout; `scripts/collect_results.py` calls `read_namespaced_pod_log` for each. Invalid rows for shards 0–7 were **12, 16, 20, 24, 28, 32, 36, 40** — **total 208**, matching the seeded ground truth.

A shared volume would have been wrong here: minikube’s default StorageClass uses the `minikube-hostpath` provisioner, which backs the volume with a *node-local* directory and supports only `ReadWriteOnce`. Six pods landed on `minikube-m02` and two on `minikube`, so the groups would have written to different directories, splitting the results silently.

With 3 nodes $\times$ 2 CPUs

I would move to **parallelism: 8**, not 6. Going $4 \rightarrow 6$ buys nothing: $\lceil 8/4 \rceil$ and $\lceil 8/6 \rceil$ are both 2 waves. Only admitting all eight at once collapses it to one, and at 500m each that costs 4000m — inside the budget.

Q4 — Deployment: self-healing and rolling updates

Self-healing

`q4/deployment.yaml` runs 2 replicas at `request = limit = 250m`, with a `readinessProbe` on `/healthz` and `maxUnavailable: 0`; `q4/service.yaml` exposes them on NodePort 30080. Deleting one pod left one against a declared two, and a replacement appeared within seconds. The responsible component is the **ReplicaSet controller** in `kube-controller-manager`: it watches pods matching `app=spam-api` and compares the count with `.spec.replicas`. This is level-triggered reconciliation, not a handler for a delete event — it observed $1 < 2$ and created one. `kubectl describe rs` attributes the **SuccessfulCreate** event to `replicaset-controller`, and the ReplicaSet name is unchanged: self-healing refills the existing one.

Rolling update

`kubectl set image` changed the pod template hash, so the Deployment created a *second* ReplicaSet and shifted replicas across: `spam-api-6c679fbdbd (v1)` to 0, `spam-api-85688f7799 (v2)` to 2, each new pod admitted only after its readiness probe passed. A probe polling the NodePort five times a second recorded **256 of 257 requests returning HTTP 200** — **99.61% availability**, version interleaving 1.0.0 and 2.0.0 before settling. The one failure came during the last old pod’s termination: endpoint removal propagates asynchronously, so the Service briefly still routes to a pod that has begun shutting down — a `preStop` hook would fix it. I report the measured figure rather than lowering the sampling rate until the gap disappeared.

Why a Deployment, not a Job

A Job’s pod template is immutable, so `set image` is rejected outright, and once its pods exited 0 the Job would be **Complete** and would never have replaced the deleted pod.

A Deployment encodes “*N* pods should be Ready, indefinitely”: success is a steady state, so restarting and version-swapping behind a readiness-gated Service are correct. A Job encodes “this finite set of work must each succeed once”: success is termination, and rolling updates are meaningless with no traffic to preserve.